

## Задание 1. Фрагмент «Руководства системного программиста» (Раздел 2. Структура программы)

Документ разработан в соответствии с требованиями ГОСТ 19.503-79 «Единая система программной документации. Руководство системного программиста. Требования к содержанию и оформлению».

### АННОТАЦИЯ

Настоящий документ представляет собой фрагмент «Руководства системного программиста» на сайт кафедры информационных и коммуникационных технологий РГПУ им. А. И. Герцена. Документ предназначен для системных программистов, осуществляющих сопровождение, настройку и эксплуатацию сайта.

## 2. СТРУКТУРА ПРОГРАММЫ

### 2.1. Общие сведения о структуре

Программа представляет собой официальный веб-сайт кафедры, функционирующий под управлением системы управления содержимым **Grav CMS**. Архитектура системы является *flat-file*, что исключает использование реляционных баз данных (вся информация хранится в виде файлов). Сайт развернут внутри контейнеров **Docker** под управлением оркестратора **Kubernetes** и доступен по адресу: <https://herzen.spb.ru>.

### 2.2. Составные части программы

Программа состоит из четырех основных логических блоков, привязанных к файловой структуре:

#### 1. Ядро Grav (каталог `/system/`):

- Маршрутизация URL-запросов.
- Парсинг текстового контента Markdown в HTML через компонент *Parsedown* (pp. 4, 20).
- Шаблонизация интерфейса на базе движка *Twig*.
- Многоуровневое кэширование данных с помощью *Doctrine Cache*.
- Обработка системных событий и консольный CLI-интерфейс (*Symfony компоненты*).

#### 2. Пользовательская область (каталог `/user/`):

- `accounts/` — учетные записи администраторов и редакторов в формате YAML.
- `config/` — конфигурационные файлы системы, плагинов и тем.
- `pages/` — текстовый контент разделов сайта в формате Markdown.
- `plugins/` — каталог расширений функционала (более 50 плагинов).
- `themes/ict/` — активная тема оформления и шаблоны сайта.

#### 3. Модули интеграции с Google Cloud Platform:

- `oauth2callback.php` — модуль авторизации пользователей через Google OAuth 2.0.
- `pr_gca_save_callback.php` — обработчик вебхуков для импорта данных из Google Forms в файл `gca.json`.
- `pr_gda_save_callback.php` — скрипт привязки страниц сайта к папкам Google Drive.
- `pr_gd_save_callback.php` — модуль автоматического скачивания и конвертации файлов с Google Диска.
- `client_id.json` — секретные ключи авторизации приложения в Google Cloud.

#### 4. Инфраструктурные компоненты:

- `Dockerfile` — инструкция для автоматической сборки Docker-образа приложения.
- `deploy.yaml` — манифест развертывания сети и контейнеров в кластере Kubernetes (описание Ingress, Service, Deployment).

### 2.3. Связи между составными частями

Цепочка обработки пользовательского запроса выглядит следующим образом:

1. Входящий HTTP-запрос от пользователя поступает на **Kubernetes Ingress** (домен `ict.herzen.spb.ru`).
2. Ingress перенаправляет трафик на внутреннюю службу **Kubernetes Service (apps-ict)** на порт 80.
3. Служба балансирует и направляет запрос непосредственно в целевой **Docker-контейнер** с веб-сервером и Grav CMS.
4. Движок Grav считывает глобальные настройки из файла `/user/config/system.yaml` и определяет активную тему оформления `ict`.
5. По URL-адресу система находит нужный файл разметки в папке контента `/user/pages/`.
6. Текст из формата Markdown обрабатывается активными плагинами и рендерится в готовый HTML-код с помощью шаблона *Twig* из каталога темы.
7. Итоговый HTML-ответ возвращается в браузер пользователя.

### 2.4. Связи с другими программами

Взаимодействие с внешними программными комплексами осуществляется по стандартным протоколам:

- **Google OAuth 2.0:** Протокол HTTPS. Предназначен для безопасной аутентификации пользователей на сайте.
- **Google Forms:** Протокол HTTP GET (Webhooks). Передает заполненные данные из форм внешних опросов на сайте.
- **Google Drive API & Calendar API:** Протокол REST API / HTTPS. Используется для чтения календаря мероприятий и скачивания/конвертации файлов.
- **Docker Engine & Kubernetes API:** Внутренние интерфейсы управления (Container API, HTTP/HTTPS). Обеспечивают оркестрацию, масштабирование и изоляцию приложения.

- **SMTP-сервер:** Протокол SMTP. Используется плагином email для отправки системных уведомлений на почту.

## 2.5. Организация хранения данных

Ввиду flat-file архитектуры все данные хранятся на жестком диске сервера без использования СУБД (MySQL/PostgreSQL) в следующих форматах:

- **Markdown (.md):** Текстовое наполнение страниц и разделов (каталог /user/pages/).
- **YAML (.yaml):** Конфигурации сайта, параметры плагинов и профили пользователей (/user/config/, /user/accounts/).
- **JSON (.json / .txt):** Динамические системные данные, вебхуки от Google Forms, настройки папок Google Drive и временные токены доступа.
- **.log:** Логирование ошибок и предупреждений сервера (/user/data/logerrors/).

## Задание 2. Список используемого программного обеспечения и технологий

### 1. Базовое системное ПО и окружение

- **Linux (Ubuntu/Debian/CentOS):** Операционная система хоста для развертывания веб-серверов.
- **Apache / Nginx:** Веб-сервер, выступающий в роли фронтенда для обработки входящих HTTP-запросов.
- **PHP (версии 7.3.6 и выше):** Среда исполнения серверного кода, на которой написан движок сайта.
  - *Расширения PHP:* curl (запросы к API Google), json (парсинг ответов), mbstring (кодировки), openssl (шифрование и OAuth), gd (сжатие картинок), zip (архивация).
- **Docker:** Платформа контейнеризации. Изолирует веб-приложение со всеми зависимостями в один легковесный контейнер.
- **Kubernetes:** Среда оркестрации. Управляет жизненным циклом контейнеров, обеспечивает автоматическое масштабирование, маршрутизацию трафика (через Nginx Ingress) и отказоустойчивость сайта.
- **Git / GitHub:** Система контроля версий. Необходима для фиксации изменений кода и синхронизации сайта с репозиторием.

### 2. Компоненты CMS-платформы

- **Grav CMS (версии 2.x):** Основная система управления контентом класса Flat-File.
- **Twig Templating:** Компонент ядра для гибкого конструирования UI-шаблонов сайта.
- **Parsedown:** Внутренний парсер для трансляции разметки Markdown в валидный HTML-код.
- **Doctrine Cache:** Модуль кэширования для ускорения работы сайта при высоких нагрузках.
- **Symfony Components (Event Dispatcher, Console):** Компоненты для обработки внутренних событий системы и управления сайтом через терминал.

### 3. Ключевые функциональные плагины Grav

- `admin / admin-addon-user-manager` — графическая панель администратора для управления контентом и пользователями.
- `login / form / email` — модули для авторизации на сайте, обработки веб-форм обратной связи и отправки писем.
- `langswitcher` — плагин для реализации многоязычного интерфейса (русский / английский).
- `feed` — автоматическая генерация новостных RSS-ленте.
- `highlight / mathjax` — плагины для красивого форматирования программного кода и отображения математических формул в формате LaTeX.
- `leaflet / lightslider` — интеграция интерактивных карт (с корпусами РГПУ) и создание слайдеров/каруселей изображений.
- `events-manager` — специализированный модуль для ведения календаря и управления мероприятиями кафедры.
- Расширения `markdown-*` (`color`, `collapsible`, `details`, `notices`) — плагины для добавления цветного текста, спойлеров, иконок FontAwesome и блоков уведомлений прямо через Markdown.